



School of Arts and Sciences
Department of Computer Science & Mathematics

Smart Campus Management System

Your Gateway to a Smarter Campus



Smart Campus

By TEK “*where Technology Empowers Knowledge*”

- **Abdallah Elrizz: 202301345**
- **Assil Halawi: 202300403**
- **Rami Assi: 202400014**
- **Lynn Akil: 202303279**
- **Nour El Itani: 202303932**

Submitted to Dr. Ramzi Haraty in fulfillment of the Phase IV project
requirements for the course CSC 490 – Software Engineering
October 2025

Table of Contents

I.	Preface.....	4
II.	Introduction.....	6
III.	Glossary	7
IV.	User Requirements	13
	A. Student Requirements	13
	Account & Access	13
	Booking and Campus Services.....	13
	Issue Reporting.....	14
	B. Faculty Staff Requirements	14
	Account & Access	14
	Booking & Teaching Support	14
	Issue Reporting & Escalation	14
	Communication & Coordination	15
	C. Maintenance & Facility Staff Requirements	15
	Issue Management.....	15
	Routine & Preventive Maintenance:	15
	Cleaning & Facility Readiness:	16
	D. System Administrator Requirements	16
V.	System Architecture	17
	A. Layered Architecture.....	17
	1. Presentation Layer	18
	User Interface & Experience (UI/UX) :.....	18
	Web Pages & Reusable Components.....	18
	Event Handling.....	18
	Validation & Error Handling	18
	Testing & Maintenance	19
	2. Application Layer	19
	Core Logic	19
	API Endpoints & Integrations	19
	Authentication & Security	19
	Background & Automated Tasks.....	20
	Error Handling & Validation	20
	Testing & Maintenance	20
	3. Data Layer	20
	Cloud Database	Error! Bookmark not defined.
	External Services.....	21
	Caching & Optimization.....	21
	Data Security & Validation.....	21

Backup & Recovery	21
Error Handling & Maintenance	21
4. Flow of Data Through Layers	22
B. Client-Server Architecture	23
1. User Interaction (Client-Side)	23
2. Client Sends API Request to Server	23
3. Backend Processes Request (Server-Side)	23
4. Database Interaction (Data Layer)	24
5. Response Sent Back to Client	24
6. Client Updates Interface (Frontend)	24
VI. System Requirements Specification	26
A. Functional Requirements	26
1. User Operations:	26
2. Faculty Operations:	29
3. Maintenance & Facility Operations:	30
4. Admin Operations:	31
B. Non-Functional Requirements	32
1. Performance Requirements:	32
2. Security Requirements:	33
3. Usability Requirements:	33
4. Availability Requirements:	33
5. Disaster Recovery Requirements:	33
VII. System Model	34
A. DFD (Level 0 – Context Diagram)	34
1. Detailed Description of the Level 0 DFD (Context Diagram)	34
a. External Entities (Actors)	34
b. Smart Campus System (Central Process)	34
B. DFD (Level 1 – Detailed Process Flow)	35
1. Detailed Explanation of Components	35
a. Booking Management	35
b. Issue Management	36
c. Events & Notifications	36
d. User Management	36
e. Authentication & Integration Services	36
f. Notification Service	Error! Bookmark not defined.
C. Class Diagram Representation	37
1. Key Components and Relationships	37
VIII. Testing & Validation:	39
A. Test Suite for Functional Requirements	39
Table 01: Login with valid credentials	39

Table 02: Login with invalid credentials	40
Table 03: Forgot password.....	41
Table 04: Create new account	42
Table 05: Book available resource	43
Table 06: Prevent double booking.....	44
Table 07: Cancel existing booking.....	45
Table 08: Waitlist unavailable resource	46
Table 09: Submit issue report	47
Table 10: Attach file to issue	48
Table 11: Update issue status.....	49
Table 12: Notify user on issue closure	50
Table 13: Chatbot valid response	51
Table 14: Chatbot fallback response	52
Table 15: Faculty override student booking	53
Table 16: Faculty post announcement.....	54
Table 17: View faculty announcements	55
Table 18: Update facility status	56
Table 19: Reopen Maintained Facility	57
Table 20: Approve or Reject Booking	58
Table 21: Dorm Booking and Cancellation	59
Table 22: Transportation Schedule and Booking	60
Table 23: Event Ticket Reservation	61
Table 24: Admin View Activity Logs	62
B. Test Suite for Non-Functional Requirements	63
Table 25: System response time	63
Table 26: Load Handling	64
Table 27: Data encryption.....	65
Table 28: Admin multi-factor authentication	66
Table 29: Access control	67
Table 30: UI consistency	68
Table 31: Mobile responsiveness	69
Table 32: Daily backups	70
Table 33: System Recovery	71
Table 34: System uptime	72
Table 35: Scalability test.....	73
Table 36: Cross-browser compatibility	74
IX. System Evolution	75
X. Appendices.....	76
A. Minimum Hardware Requirements.....	76
B. Application Dependencies	77

C.	Database Schemas	79
1.	USER	79
2.	DEPARTMENT.....	79
3.	RESOURCE	79
4.	BOOKING_REQUEST	79
5.	BOOKING.....	80
6.	EVENT	80
7.	EVENT_TICKET	80
8.	DORM_UNIT	81
9.	TRANSPORTATION_ROUTE.....	81
10.	PARKING_SPACE	81
11.	ISSUE_REPORT.....	82
12.	ISSUE_ACTIVITY.....	82
13.	CLEANING_TASK	82
14.	TUTORING_SESSION.....	82
15.	STUDENT_EMPLOYMENT	83
16.	EMPLOYMENT_APPLICATION	83
17.	NOTIFICATION	83
XI.	Index.....	89

I. Preface

This report presents a comprehensive overview of the Smart Campus System, a software solution designed to enhance campus life through intelligent resource management and digital automation. The purpose of this document is to outline the main aspects of the system's

development, including its user requirements, system specifications, and its expected evolution to meet future university needs. The report is organized into several key sections.

The Introduction provides context for the project and defines its overall objectives.

The Glossary clarifies technical terms to ensure consistency and understanding among all readers.

The User Requirements section specifies the needs and expectations of each stakeholder group, while the System Requirements section details the functional and technical components necessary for system implementation.

The System Architecture section presents a high-level overview of the system's structure, illustrating how its core functions are distributed across different modules and components. It also identifies reused architectural elements that contribute to development efficiency and maintainability.

The System Models section provides visual representations of the system's internal and external relationships, including data-flow, object, and interaction models, to clarify how components communicate and operate within the campus environment.

The Testing and Validation section outlines the complete test suite used to evaluate both functional and non-functional requirements. This includes detailed test cases developed to verify system performance, usability, and reliability, ensuring that the Smart Campus System meets all design objectives.

The System Evolution section discusses how the system is able to adapt to future changes in hardware or user needs.

The document concludes with Appendices, offering supplementary information relevant to the development and design of the Smart Campus System such as hardware and database description.

This report is intended for software developers, project stakeholders, and academic management personnel interested in the technical and functional design of the Smart Campus platform. By documenting each phase of the software engineering process, this report provides a structured and transparent view of the system's foundation, ensuring clarity for future development and expansion.

II. Introduction

University campuses often face challenges in efficiently managing facilities, services, and student requests. Tasks such as booking classrooms, reporting equipment failures, managing dorms, and coordinating campus events are often handled through disconnected systems or manual processes, leading to delays, confusion, and resource conflicts.

To address these challenges, Smart Campus System has been developed as an integrated platform that centralizes all essential campus services within one system. The platform enables students and faculty to book classrooms, labs, and sports halls, report maintenance issues, reserve dorm rooms, access transportation, organize or attend events, and other campus resources seamlessly. The system also incorporates AI-driven features, such as a chatbot for instant support to answer FAQs. These intelligent tools aim to improve service efficiency, reduce administrative workload, and enhance the overall campus experience.

By combining automation, real-time data updates, and user-friendly interfaces, the Smart Campus System creates a unified digital ecosystem where all stakeholders—students, faculty, maintenance staff, and administrators—can interact efficiently. The system aligns with the university’s strategic objective of promoting digital transformation, sustainability, and operational excellence in higher education.

The goal is to ensure a clear, structured foundation for development and future enhancements, supporting a smarter and more connected university environment.

III. Glossary

1. AJAX (Asynchronous JavaScript and XML):

A web development technique that allows webpages to update content asynchronously, meaning parts of a page can refresh without reloading the entire site.

2. API (Application Programming Interface):

A structured set of rules and endpoints that enable software components or systems to communicate and exchange data securely.

3. Application Layer:

The system layer that processes business logic, handles data flow, validates user input, and coordinates communication between the user interface and the database.

4. Automation:

The use of technology to perform repetitive or scheduled tasks automatically without human intervention, improving speed and accuracy.

5. Backup:

A copy of important system data stored securely to allow recovery in case of hardware failure, deletion, or corruption.

6. Bootstrap:

A front-end web framework that simplifies creating responsive and mobile-friendly interfaces using pre-designed components and layouts.

7. CI/CD Pipeline (Continuous Integration/Continuous Deployment):

An automated process that integrates code changes frequently, tests them, and deploys updates efficiently to production environments.

8. Client–Server Architecture:

A network model in which the client (user interface) sends requests to a central server, which processes them and returns results or data.

9. Cloud-Based Deployment:

A method of hosting applications on remote internet servers, allowing on-demand access, scalability, and maintenance from anywhere.

10. Core Logic:

The set of rules and operations that define how the system behaves, processes data, and enforces business rules.

11. CSS (Cascading Style Sheets):

A language used to style HTML content by controlling layout, colors, fonts, and overall design of webpages.

12. Dashboard:

A visual interface where users or administrators can view system data, manage bookings, and track reports in one centralized view.

13. Data Encryption:

The process of converting readable data into coded form to prevent unauthorized access during transmission or storage.

14. Data Flow Diagram (DFD):

A graphical model that illustrates how data moves through the system, showing inputs, processes, and outputs.

15. Data Layer:

The system layer responsible for data storage, retrieval, and management in databases or external services.

16. Database:

A structured and organized collection of data that stores user profiles, bookings, and records accessible through queries.

17. Disaster Recovery:

A contingency plan that ensures quick restoration of system operations and data after technical failures or cyber incidents.

18. Entity:

A distinct object or concept represented in the database, such as a user, booking, or event, each with unique attributes.

19. Frontend:

The part of the system that users directly interact with, built using HTML, CSS, and JavaScript to create the interface.

20. Functional Requirements:

Statements that define the core actions and operations the system must perform to meet user needs.

21. GitHub:

A version control platform used to store code repositories, manage collaboration, and automate software deployment.

22. HTML5 (Hypertext Markup Language):

The standard markup language used to structure and display content on web browsers.

23. Integration:

The process of connecting independent systems or modules so they exchange and synchronize data efficiently.

24. Interface:

The visual and functional point of interaction between the user and the system.

25. Issue Report:

A digital form that users submit to report technical or maintenance problems, often including location and photos.

26. JavaScript:

A scripting language used to add interactivity, logic, and dynamic updates to web pages.

27. JSON (JavaScript Object Notation):

A lightweight data format used for exchanging information between the client and server in structured key–value pairs.

28. Layered Architecture:

A software design model that separates a system into layers (Presentation, Application, and Data) for better organization and maintainability.

29. LocalStorage:

A browser-based feature that allows data to be stored locally on a user's device for faster retrieval.

30. Material Design:

A design framework by Google that provides guidelines for clean, consistent, and user-friendly web interfaces.

31. Module:

A self-contained unit in software that performs a specific function, such as booking or issue management.

32. Non-Functional Requirements:

Specifications that define the quality attributes of the system, such as security, performance, and usability.

33. Notification System:

A component that sends alerts or messages to users about bookings, events, or system updates.

34. NoSQL Database:

A non-relational database that stores data in flexible formats like documents or key-value pairs rather than fixed tables.

35. Postman:

A testing tool used to verify API functionality and ensure that requests and responses work as expected.

36. Presentation Layer:

The topmost layer in the system architecture responsible for user interaction, data display, and input validation.

37. RESTful API (Representational State Transfer):

A web service style that defines structured, stateless communication between client and server using standard HTTP methods.

38. Scalability:

The system's ability to handle growth in users or data without losing performance or stability.

39. Security Policy:

A set of guidelines and rules designed to protect system data from unauthorized access and misuse.

40. Serverless Computing:

A cloud computing model where backend functions run automatically without needing dedicated servers.

41. Stakeholders:

All individuals or groups involved in or affected by the system, including students, faculty, staff, and administrators.

42. Stripe:

A secure payment processing platform used for handling online transactions and verifying payment details.

43. System Administrator:

A person responsible for managing system settings, security configurations, and user permissions.

44. System Architecture:

The structural blueprint that defines how software components are organized and interact to fulfill requirements.

45. System Evolution:

The process of continuously updating and improving the system to adapt to technological and user changes.

46. Token-Based Authentication:

A security mechanism where users receive a unique encrypted token after logging in, used to verify identity in each request.

47. UI (User Interface):

The part of the software users directly interacts with, including buttons, forms, and navigation menus.

48. UX (User Experience):

The overall feeling and satisfaction a user experiences while interacting with the system.

49. Uptime:

The percentage of time the system remains available and operational without interruption.

50. Validation:

The process of checking whether user input or data meets specified rules and constraints before processing.

51. Version Control:

A software management practice that tracks and manages changes to source code over time.

52. Waitlist Mechanism:

A system feature that allows users to reserve a spot in line for unavailable resources and get notified when they become available.

IV. User Requirements

The user requirements for Smart Campus are categorized into four groups:

- a. **Student Requirements** – Defined as the requirements of students who use the system to book resources, submit limited issue reports, and access campus services.
- b. **Faculty Staff Requirements** – Defined as the requirements of faculty members who manage class-related bookings, request resources, report issues, and oversee academic usage.
- c. **Maintenance & Facility Staff Requirements** – Defined as the requirements of staff who resolve reported issues and maintain campus facilities.
- d. **System Administrator Requirements** – Defined as the requirements of IT administrators who maintain, configure, and secure the system.

A. Student Requirements

Account & Access

- Students shall be able to create and access accounts using university credentials.
- Students shall be able to log in securely and recover forgotten credentials.

Booking and Campus Services

- Students shall be able to search, reserve, and cancel bookings for study rooms, library seats, library computers, study rooms, sports halls.
- Students shall receive booking confirmations, updates, and cancellation notices.
- Students shall rent dorm rooms.
- Students shall be able to access transportation services within campus and off campus.
- Students shall be able to reserve tutoring slots.
- Students shall be able to view nursing appointments.
- Students shall be able to apply to employment opportunities on campus.
- Students shall be able to access lost-and-found.
- Students shall be able to reserve event tickets organized by faculty members.
- Students shall be able to view and reserve parking spaces.
- Students shall be able to ask questions to an AI Chatbot.
- Students shall be able to view booking history.

Issue Reporting

- Students shall be able to submit issue reports limited to equipment failures, network problems, environmental issues, and study or common area conditions.
- Students shall be able to attach photos or details such as the location of the issue or a short description.
- Students shall be able to track the status of their reports.
- Students shall be able to view history reports.

B. Faculty Staff Requirements

Account & Access

- Faculty staff shall be able to log in securely using university credentials.
- Faculty staff shall be able to view and manage their teaching schedules and assigned classrooms.

Booking & Teaching Support

- Faculty staff shall be able to request classroom reservations for exams, extracurricular activities, labs, and sports halls for teaching purposes.
 - The system shall forward these requests to the campus scheduling administrator (or department coordinator) for review and approval.
 - The system shall notify faculty when their reservation request is approved or rejected. Faculty staff shall be able to request to override student bookings in case of academic conflicts.
- Faculty staff shall be able to request specialized equipment and classroom resources required for teaching activities.
- Faculty staff shall be able to organize and manage tutoring sessions for their courses.
- Faculty staff shall be able to schedule and publish academic events, seminars, or workshops.
- Faculty staff shall receive notifications confirming booking approvals, modifications, or cancellations related to their classes.

Issue Reporting & Escalation

- Faculty staff shall be able to report urgent facility issues during class sessions

- Faculty staff shall be able to monitor the status of their submitted reports and receive updates upon resolution.
- Faculty staff shall be able to request a change in classroom if the matter of the issue escalated.

Communication & Coordination

- Faculty staff shall be able to communicate with students through announcements related to classroom bookings or tutoring sessions via an announcement button on the landing page.
 - Faculty staff should be able to edit, delete, or schedule announcements sent to fellow students.
- Faculty staff shall be able to view event schedules organized by other departments or faculty members to avoid booking conflicts.

C. Maintenance & Facility Staff Requirements

Issue Management

- Maintenance staff shall be able to receive categorized issue reports and view details including location, description and photo attached
- Maintenance staff shall be able to accept and assign issue reports to themselves directly from the issue page.
- Once an issue is accepted, the system shall automatically update its status to “In Progress” and display the assigned staff member’s name. Maintenance staff shall be able to update repair progress, provide estimated completion times, and close report.
- Maintenance staff shall be able to attach before-and-after photos or notes for documentation.
- Maintenance staff shall receive real-time notifications for newly reported or reassigned issues in their domain.

Routine & Preventive Maintenance:

- Maintenance staff shall be able to view scheduled preventive maintenance tasks assigned to them, and mark them done once finished
- Maintenance staff shall be able to report new issues identified during routine checks directly from their dashboard.

- Maintenance staff shall be able to manage lost-and-found records, including item details and retrieval status.
- Maintenance staff shall be able to generate daily summaries of completed repairs, inspections, and cleaning tasks for supervisor review.

Cleaning & Facility Readiness:

- Cleaning staff shall be able to access daily cleaning schedules and assigned areas such as classrooms, dorms, or cafeterias.
- Cleaning staff shall be able to mark cleaning tasks as completed and report any maintenance problems noticed during cleaning.
- Maintenance staff shall be able to report any unsafe or unsanitary conditions that require further attention.

D. System Administrator Requirements

- System administrators shall be able to manage user accounts and assign role-based permissions.
- System administrators shall be able to approve, suspend, or deactivate accounts violating university policies.
- System administrators shall be able to configure booking policies and system settings.
- System administrators shall be able to monitor system performance and check for errors.
- System administrators shall be able to manage integrations with other university modules such as transportation, events, and dorms.
- System administrators shall be able to enforce security measures including data backup and access control.
- System administrators shall be able to review activity logs and respond to suspicious actions.

V. System Architecture

The System Architecture of Smart Campus System defines the overall structure and organization of the system, showing how various components interact to deliver the required functionalities. It provides a high-level overview of the distribution of functions across the system's modules and illustrates how the software components are layered, connected, and managed to ensure scalability, security, and maintainability. The system is developed using two complementary architectural models:

- a. **Layered Architecture** – which describes the internal organization of the system into distinct functional layers. Each layer shall handle specific responsibilities and shall interact only with adjacent layers to maintain separation of concerns.
- b. **Client–Server Architecture** – which defines the external communication model between users (clients) and the central backend server through a secure network connection.

Note: Components reused from existing frameworks or external services are written in green.

A. Layered Architecture

The Smart Campus System follows a three-layer architecture:

- **Presentation Layer (Frontend):** This layer shall handle all user interactions, process inputs through web forms, and display dynamic data in a responsive interface. It shall serve as the bridge between the user and the system, ensuring accessibility, clarity, and a seamless user experience across devices.
- **Application Layer (Backend):** This layer shall manage the system's core logic, process and validate user requests, and coordinate communication between the frontend and the database. It shall also integrate with external services through APIs and enforce authentication, authorization, and security policies.
- **Data Layer (Database & External Services):** This layer shall be responsible for data storage, retrieval, and external integrations. It shall ensure data integrity, security, and reliable synchronization across the system.

Each layer shall operate independently but interact with adjacent layers through well-defined interfaces.

1. Presentation Layer

- **User Interface & Experience (UI/UX) :**

The web interface shall emphasize responsiveness, accessibility, and visual consistency across all devices. It shall follow Google's Material Design principles to ensure a clean layout, intuitive navigation, and meaningful animations. The structure of the webpage shall be defined using HTML5 for semantic content, styled with CSS, and made interactive with JavaScript. It shall utilize Bootstrap or Material Design for Web to implement responsive grids and ready-made UI components. Responsiveness shall be achieved through CSS Flexbox and Grid, enabling automatic resizing and real-time adaptation to desktops, tablets, and smartphones.

- **Web Pages & Reusable Components**

The Presentation Layer shall be divided into functional web pages for each user role:

- Student: Booking resources, reporting issues, viewing events.
- Faculty: Booking resources, reporting issues, publishing events, monitoring departmental activities.
- Maintenance and Facility: Managing issues, routine and preventive maintenance.
- Admin Dashboard: Managing users, approving bookings, reviewing activity logs, and monitoring system performance.

Each portal shall reuse common UI components (navigation bar, footer, search bar, forms) to ensure visual consistency and ease of maintenance.

- **Event Handling**

User interactions (form submissions, button clicks, filter changes) shall be managed through JavaScript event listeners.

Data shall be exchanged asynchronously with the backend through the **Fetch API** or **AJAX**, allowing pages to update dynamically without full reloads.

- **Validation & Error Handling**

Client-side validation shall use HTML5 constraints and JavaScript regex patterns for

accurate data entry (emails, dates, IDs).

Descriptive error messages shall appear instantly to guide the user, and invalid submissions shall be blocked before reaching the server.

- **Testing & Maintenance**

Browser-based testing shall verify cross-browser compatibility (Chrome, Edge, Firefox, Safari).

Automated tools such as **Cypress** shall be used to test form behavior and UI responsiveness.

2. Application Layer

- **Core Logic**

Core business logic — including booking validation, issue routing, event management, and notification handling — is implemented in modular controller and service files within the Node.js + Express backend.

These components ensure consistent enforcement of system rules, such as checking classroom availability, validating faculty approvals, and verifying user permissions across all modules.

- **API Endpoints & Integrations**

The backend shall communicate with the frontend entirely through RESTful API endpoints built using Express.js.

Endpoints such as /api/bookings, /api/issues, /api/events, and /api/users exchange data in JSON format over secure HTTP connections.

- **Authentication & Security**

Authentication and authorization are handled using:

- MySQL database for storing users & roles
- bcrypt to securely hash passwords
- JWT (JSON Web Tokens) to verify identity on every request

After login, the backend generates a JWT token, which the frontend includes in every request.

The backend verifies the token before allowing access.

Invalid or missing tokens cause the request to be rejected.

This ensures secure, role-based access across the entire system.

- **Background & Automated Tasks**

Some automated processes are handled directly through backend logic, such as:

- Sending booking confirmations
- Preventing resource conflicts
- Updating logs
- Assigning issue priorities
- These operations run inside controller logic without impacting frontend performance.

- **Error Handling & Validation**

All inputs and requests undergo strict validation before processing.

- If an error occurs, the backend:
- Sends a clear JSON error message
- Logs the issue for debugging
- Ensures the request does not corrupt data
- This helps maintain system reliability and easier troubleshooting.

- **Testing & Maintenance**

All backend endpoints were tested using:

- Postman
- Browser-based testing through the frontend
- Regular maintenance includes performance optimization, SQL query tuning, and ensuring API responses are consistent with frontend expectations.

3. Data Layer

- **Relational Database (MySQL):**

- The system uses a relational MySQL database hosted on Railway.
- Data is exchanged via SQL queries executed from the backend.

- There is no real-time synchronization; instead, the frontend requests updated data through API calls.
- **External Services**

The only external service used is:

 - Railway MySQL host (for database)
 - Railway backend host (for backend runtime)

This ensures simple, scalable deployment.
- **Caching & Optimization**

To reduce load times:

 - Frequently accessed data (e.g., room lists, event schedules) is cached in backend memory or fetched selectively through optimized SQL queries.
 - SQL indexing is used to optimize performance on large tables.
- **Data Security & Validation**

Security measures include:

 - Access controlled through JWT
 - Hashed passwords stored in MySQL
 - Input validation on all endpoints
 - SQL injection prevention using prepared statements (mysql2)
 - These measures ensure safe read/write operations.
- **Backup & Recovery**

Railway provides tools for:

 - Manual or scheduled database backups
 - Exporting .sql snapshots
 - Restoring from backups when necessary
 - This guarantees data recovery in case of failure.
- **Error Handling & Maintenance**

If a database operation fails:

 - The backend returns a meaningful JSON error message
 - Logs the issue internally
 - Ensures no partial writes occur

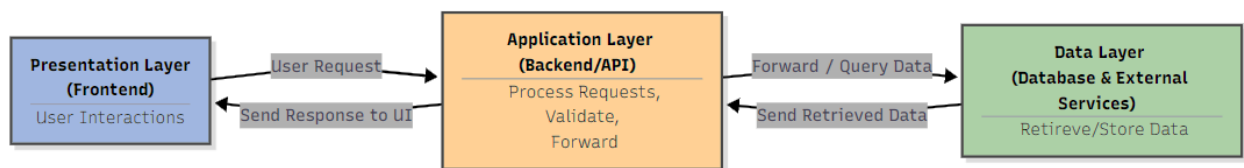
- Regular maintenance includes testing backup procedures and monitoring SQL query performance.

4. Flow of Data Through Layers

User interactions captured at the Presentation Layer shall be processed by the Application Layer.

Requests shall be validated and forwarded to the Data Layer for retrieval or storage.

The resulting data shall then be returned to the Presentation Layer for display, ensuring a smooth, secure, and efficient flow of information throughout the Smart Campus System.



B. Client-Server Architecture

The SmartCampusSystem operates on a client–server model, where the client (browser) interacts with the backend server hosted on Railway. The server processes logic, validates data, and communicates with the MySQL database.

1. User Interaction (Client-Side)

The user shall interact with the SmartCampusSystem website through a web browser. Depending on the role (Student, Faculty, Maintenance, or Admin), users shall perform actions such as booking resources, reporting issues, viewing events, or approving requests.

Example:

A student logs into SmartCampus through the login page.

The frontend:

- Collects input through HTML forms
- Validates fields with JavaScript
- Sends clean, structured requests to the backend
- Displays success/error messages to the user
- Client Sends API Request to Server through Fetch API

2. Client Sends API Request to Server

When a user performs an action, the frontend sends an HTTP request (GET, POST, PUT, DELETE) to the backend API.

Example:

- POST /api/bookings

The request includes:

- JSON body
- The JWT authentication token
- Role information
- Communication happens over secure connections.

3. Backend Processes Request (Server-Side)

The backend:

- Verifies JWT token
- Checks MySQL for availability or conflicts
- Applies system rules
- Writes or retrieves data from MySQL

- Sends a JSON response back to the client

Example:

If the room is unavailable → Sends a rejection message.

If available → Saves the booking + returns confirmation.

4. Database Interaction (Data Layer)

Backend interacts with MySQL to:

- Insert bookings, issues, events
- Update statuses
- Retrieve data for dashboards
- There is no real-time syncing; clients request new data as needed.
- Media uploads (if any) are stored on the backend server or uploaded through custom endpoints.

5. Response Sent Back to Client

Once the database operation is complete, the backend shall generate a structured JSON response and shall send it back to the client over the same secure connection.

The response shall include confirmation details such as booking ID, room name, and status (*“Pending Approval”*).

If any error occurs, such as invalid data or connection timeout, the server shall send an error response describing the issue, allowing the frontend to display the correct message to the user.

Example:

- If the booking succeeds: “Your booking has been submitted and is awaiting approval.”
- If it fails: “Booking could not be processed. Please try again later.”

6. Client Updates Interface (Frontend)

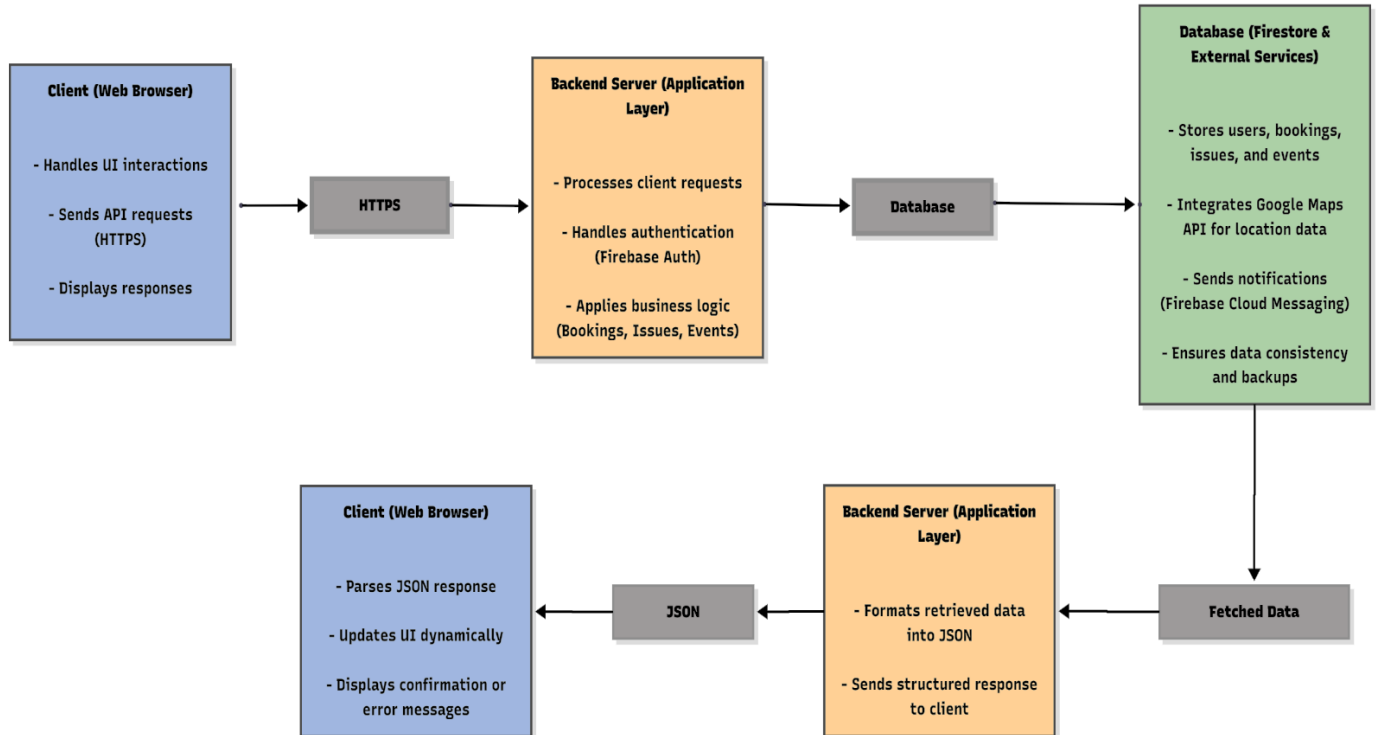
After receiving the server’s response, the browser shall update the user interface dynamically using JavaScript.

- Successful bookings shall trigger a confirmation message.

- After the booking is confirmed, the user's booking history shall be updated.
- If an error occurs, the system shall display a clear notification, permitting the user to adjust and resubmit.

Example:

The student sees a message like *“Room C-203 successfully booked for Monday, 10 AM”*



VI. System Requirements Specification

A. Functional Requirements

1. User Operations:

Home Interface:

- The system shall display a landing page with the description of Smart Campus System and its available services.
- The system shall display a form to allow users to either:
 - Create an account using university credentials.
 - Log in using existing credentials.
- The system shall provide quick-access links to core modules: Bookings, Issue Reporting, Events and Transportation.
- The system shall provide password recovery assistance through two options:
 - A “Forgot Password” button that initiates the password reset process.
 - A contact link to the IT Department for manual assistance if login issues persist.

User Profile:

- If needed, the system shall allow users to create an account by providing the following required fields:
 - First and last name
 - University email address
 - University ID number
 - Faculty or department
 - Role (student or faculty)
 - Password
- The system shall enforce the following password constraints:
 - At least one uppercase letter
 - At least one special character
 - At least one number
 - Minimum length of eight characters
- The system shall allow users to update their personal information while complying with security constraints.

- The system shall allow users to recover forgotten credentials securely.

Campus Services & Booking Management:

- The system shall provide a centralized interface where users can view and access campus services and resources.
- The system shall display all service categories with descriptions, availability status, and quick-access buttons for reservation or request.
- The system shall allow users to reserve/request reservations in all kinds of available resources and services including:
 - View and reserve classrooms, laboratories, and sports halls for academic or recreational purposes.
 - Browse and book library seats and computer stations based on real-time availability.
 - Request or rent dorm rooms and view occupancy or duration details.
 - Schedule tutoring sessions with available instructors or teaching assistants.
 - Book nursing appointments and view service hours.
 - Apply for on-campus employment opportunities.
 - Access lost-and-found services to report or claim lost items.
 - View and reserve tickets for campus events, seminars, and workshops.
 - Manage parking reservations, including viewing space availability and permit status.
- The system shall allow users to search for services using filters such as:
 - Service type such as booking, dorm, event, transportation and more.
 - Availability by date, time, or occupancy.
 - Location (building, department, or campus area).
 - Resource type (room, seat, computer, parking, or equipment).
- The system shall provide a waitlist mechanism for fully booked resources:
 - Users shall be able to add unavailable rooms, dorms, books, or seats to their waitlist.
 - The system shall notify users automatically when any wish list item in their list becomes available due to a cancellation or expiration of their reservation period.

- Users shall confirm the newly available booking within a limited time window before it passes to the next user.
- The system shall allow users to modify or cancel their bookings within the defined cancellation policy.
- The system shall prevent double booking of the same resource for overlapping time slots.
- The system shall send booking confirmations, cancellations, or modification messages via in-app notifications and email.
- The system shall send reminders for upcoming reservations one hour before the scheduled time.
- The system shall allow users to view booking history through their profile page.

Transportation Services:

- The system shall provide on-campus transportation features, including:
 - View available campus shuttle schedules for on-campus routes.
 - Reserve seats on available shuttles by selecting a route, date, and time slot.
 - Cancel or modify their shuttle reservations.
- The system shall provide on-campus transportation features, including:
 - Reserve seats on university bus services.
 - View bus capacity, departure times, and estimated arrival times.
 - Receive confirmation and reminder notifications before scheduled departure.
- The system shall notify users of shuttle or bus delays, cancellations, or route changes through in-app alerts and email notifications.
- The system shall allow administrators to update transportation schedules, vehicle capacities, and route information dynamically

Issue Reporting:

- The system shall allow users to submit issue reports by providing the following mandatory fields:
 - Issue category (maintenance, IT, environmental, or general).
 - Location of the issue (building, room, or area).
 - Description of the problem.

- The system shall allow users to optionally attach supporting materials such as photos, documents, or short text notes.
- The system shall generate a unique issue ID for tracking purposes starting from ID = 0.
- The system shall categorize each report automatically based on keywords and forward it to the relevant department.
- The system shall notify the maintenance department immediately upon issue submission.
- The system shall allow users to track the progress and resolution status of submitted issues.

AI Chatbot:

- The system shall provide an AI chatbot to answer frequently asked questions related to campus services (bookings, events, transportation, dorms).
- The chatbot shall operate 24/7 and respond instantly to user inquiries.
- The system shall allow the chatbot to redirect users to appropriate staff when an inquiry cannot be resolved automatically.
- The system shall monitor chatbot performance and notify administrators or IT staff when the service shows repeated errors or poor response quality

2. Faculty Operations:

Teaching & Booking Management:

- The system shall allow faculty members to request reservations of classrooms, laboratories, and sports halls for academic sessions or special activities.
- The system shall allow faculty to request an override of any students' bookings in the event of academic conflicts, subject to administrative policy rules.
- The system shall allow faculty to request a reservation of specialized teaching equipment or classroom resources such as projectors or lab instruments.
- The system shall allow faculty to organize and manage tutoring sessions linked to their courses.
 - Session topic or course.
 - Tutor name or assigned teaching assistant.

- Schedule and room location.
- The system shall allow faculty to create, publish, and manage academic events, seminars, or workshops through the events section.
 - Event title, description, and department.
 - Date, time, and venue.
 - Capacity limit and registration rules.
- The system shall send automated notifications to faculty when bookings or event reservations are approved, modified, or canceled.
- The system shall allow faculty to view classroom and resource availability before requesting to confirm on new bookings.
- The system shall prevent conflicting faculty bookings within the same time frame or classroom.
- The system shall display event schedules from other departments to help faculty avoid booking conflicts.

Issue Reporting & Escalation:

- The system shall allow faculty to report urgent issues during class sessions.
 - Issue type (technical, maintenance, environmental).
 - Location (building name, room number).
 - Description of the issue.
- The system shall send immediate notifications to IT or maintenance staff when a critical issue is reported.
- The system shall allow faculty to monitor the progress and resolution of reported issues.
- The system shall allow faculty to request a temporary classroom change if recurring issues occur in a specific location.
- The system shall notify administrators when a classroom change request is submitted for review and approval.

3. Maintenance & Facility Operations:

Home Interface:

- The system shall display a home interface showing assigned maintenance tasks, active issue reports, and facility status updates.
- The interface shall provide quick-access options to update issue progress; mark repairs as completed and report new problems.

Issue Management:

- The system shall provide maintenance staff with access to all reported issue tickets categorized by type and urgency.
- The system shall allow maintenance staff to view issue details, update progress, attach repair notes or photos, and close completed tickets.
- The system shall send automatic notifications to the reporting user once an issue is resolved.

Facility Status Management:

- The system shall allow maintenance staff to mark facilities (dorms, gym, cafeteria, classrooms) as Active, Under Maintenance, or Closed.
- The system shall record facility status updates with timestamps for administrative tracking.
- The system shall allow maintenance staff to manage lost-and-found records in their assigned facilities.

4. Admin Operations:

Service & Resource Management:

- The system shall provide administrators with an interface to:
 - Add new campus services and resources such as rooms, labs, sports halls, events, transportation routes, parking.
 - Update existing service/resource details such as availability, capacity, rules, descriptions.
 - Remove or temporarily disable services/resources that are unavailable.
 - Administrators, when needed, shall be responsible for reviewing and approving faculty and student bookings and override requests in accordance with university policies

- The system shall log management actions, including:
 - Administrator's ID
 - Date and time of action

System Configuration & Monitoring:

- The system shall allow administrators to configure booking rules (time limits, cancellation policies, and buffer periods).
- The system shall allow administrators to manage integrations with other campus modules (events, maintenance, transportation).
- The system shall allow administrators to monitor system health, uptime, and activity logs.
- The system shall allow administrators to configure and update AI chatbot modules.

Security & Compliance:

- The system shall encrypt all stored and transmitted data using strong encryption methods (such as AES-256)
- The system shall automatically back up all databases every 24 hours and store them securely in the cloud.
- The system shall keep backup copies for 30 days and allow administrators to restore data if needed.
- The system shall record and audit all administrative actions.
- The system shall notify administrators of suspicious activity or failed login attempts.

Notification System:

- The system shall send notifications for new user registrations, bookings, updates, and events.
- The system shall maintain a record of all notifications sent.

B. Non-Functional Requirements

1. Performance Requirements:

- The system shall handle a minimum of 1,000 concurrent users without degradation in performance.

- Page load times shall be under 3 seconds for standard operations.

2. Security Requirements:

- The system shall encrypt all sensitive data.
- Multi-factor authentication shall be required for administrative accounts.
- All access attempts shall be logged for auditing.

3. Usability Requirements:

- The system shall ensure ease of use, allowing key user tasks (booking, issue reporting, schedule viewing) to be completed efficiently with minimal steps and intuitive navigation.

4. Availability Requirements:

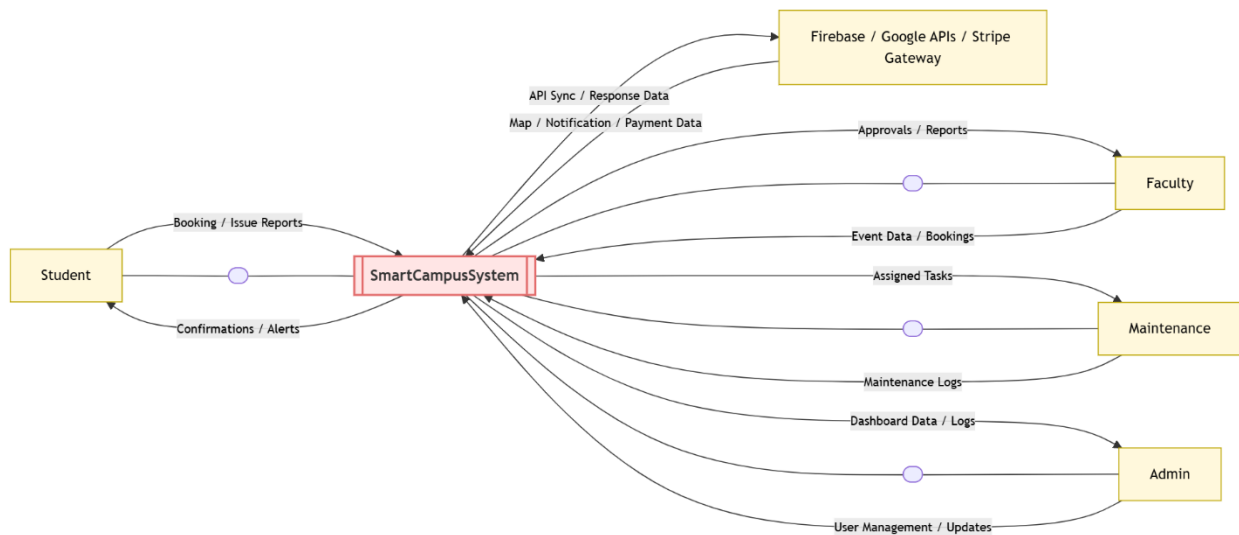
- The system shall maintain 99.5% uptime annually, excluding scheduled maintenance.

5. Disaster Recovery Requirements:

- The system shall automatically back up all data daily
- The system shall restore full functionality within 24 hours in case of system failure.

VII. System Model

A. DFD (Level 0 – Context Diagram)



Reference: mermaid.js.org

1. Detailed Description of the Level 0 DFD (Context Diagram)

a. External Entities (Actors)

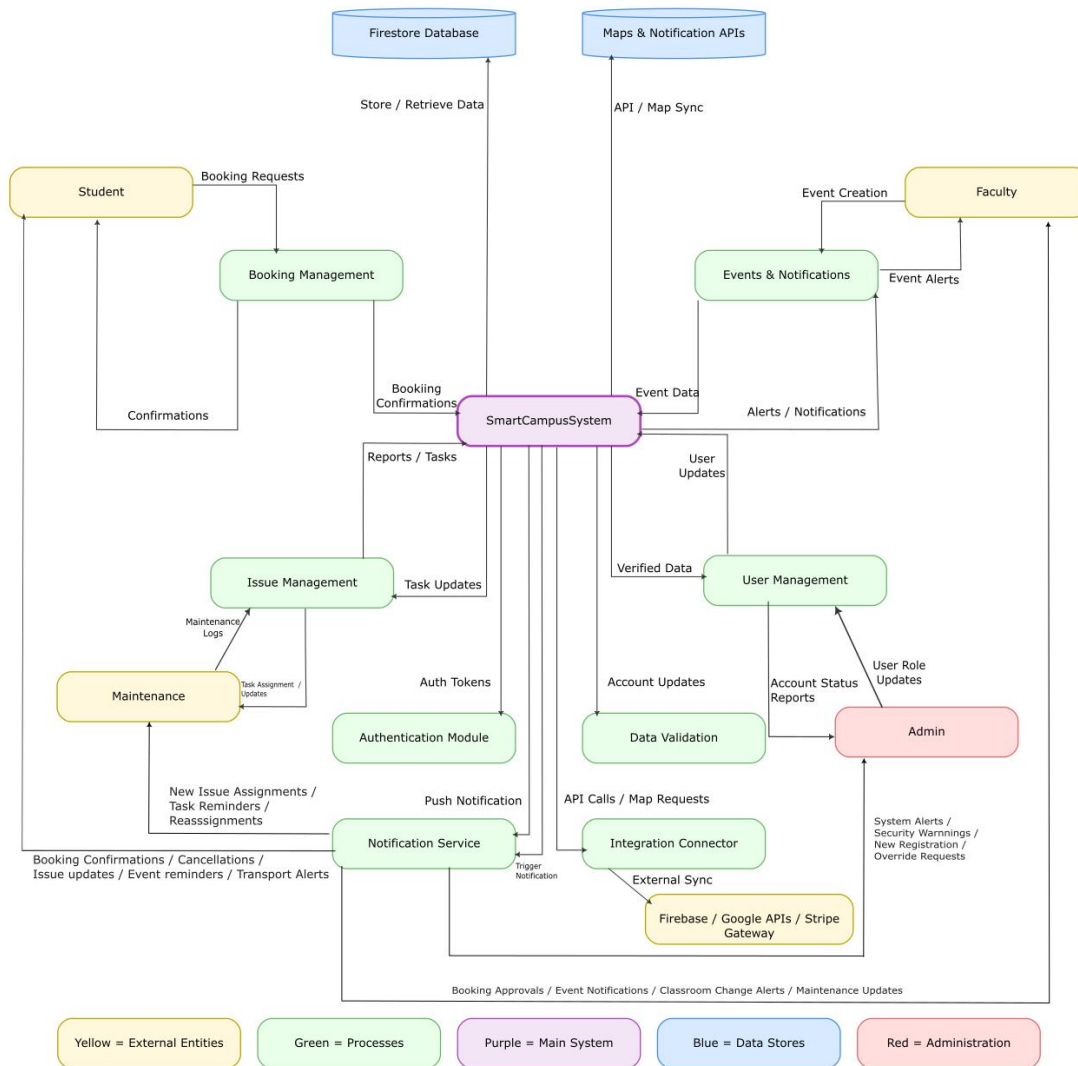
- **Student:** Uses the system to reserve classrooms, dorms, transport, report issues, and view events.
- **Faculty:** Requests class bookings, tutoring sessions, and event management.
- **Maintenance Staff:** Receives and updates issue reports, completes assigned tasks.
- **Administrator:** Manages users, approvals, and system settings.

b. Smart Campus System (Central Process)

The Smart Campus System acts as the core platform connecting all entities. It processes booking and issue requests, manages data storage, sends notifications, and

interacts with external APIs. It validates input, applies business logic, and ensures each department or staff receives the correct information in real time.

B. DFD (Level 1 – Detailed Process Flow)



Reference: inkscape

1. Detailed Explanation of Components

a. Booking Management

Handles all booking requests from students and faculty. It validates availability, stores requests in the database, and forwards them for admin approval. Once approved, it sends confirmations and reminders through the Notification Service.

b. Issue Management

Processes issue reports submitted by students or faculty. It forwards categorized issues to maintenance staff, tracks progress, and updates reporters on resolution status.

c. Events & Notifications

Manages event creation, event ticketing, and real-time alerts. It sends notifications to all roles based on activity (bookings, issues, or schedule changes).

d. User Management

Administers role-based access, account activation, and permissions. Admins use it to manage users and monitor logs.

e. Authentication & Integration Services

Authenticates users MySQL, issues secure tokens, and connects to external APIs.

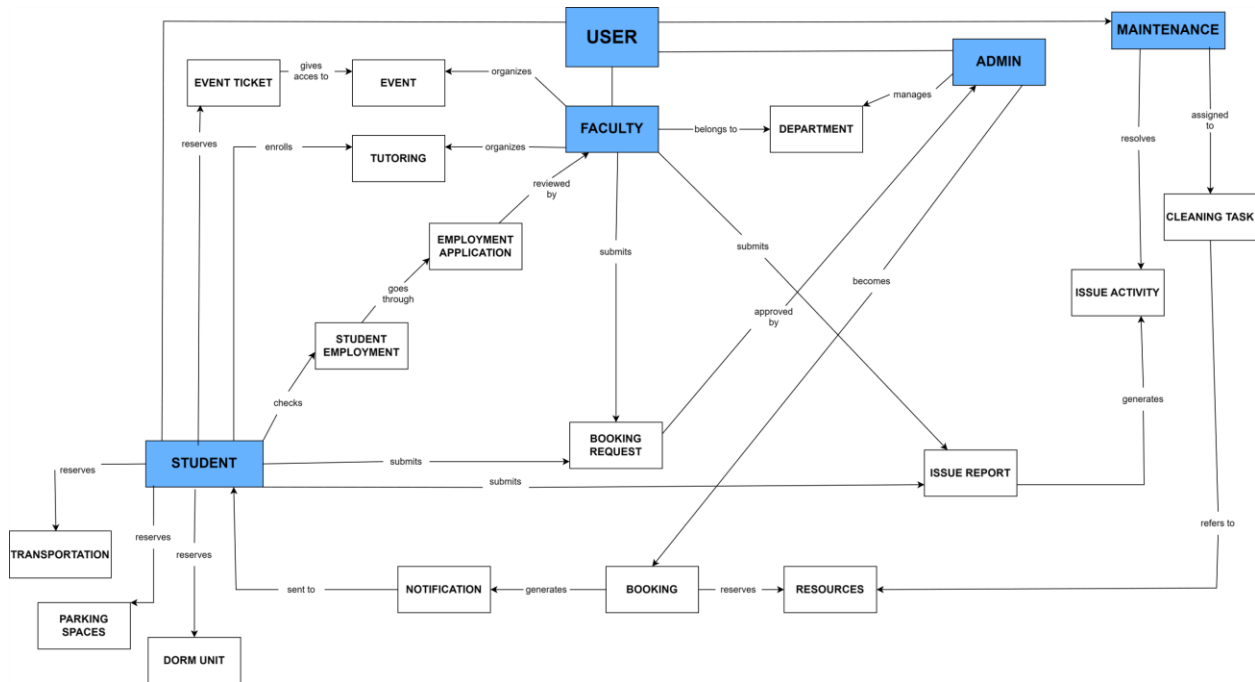
f. Railway/Vercel

To deploy the backend and frontend

g. MySQL Database

Data stores that record all bookings, issues, events, and logs. External APIs handle mapping, storage, and messaging.

C. Class Diagram Representation



Reference: draw.io

The class diagram represents the object-oriented structure of the Smart Campus System, showing how data and functionality are distributed among the system's main entities. It illustrates the relationships between users, operations, and data management components, ensuring modularity, maintainability, and a clear link between the software design and its real-world processes.

1. Key Components and Relationships

- a. The User superclass defines shared attributes and methods for all system users — Student, Faculty, Maintenance, and Admin. Each subclass inherits core functions such as authentication, role management, and notification handling.
- b. Student interacts with booking, events, and issue modules.
- c. Faculty creates and manages events, approves bookings, and supervises classroom activities.
- d. Maintenance manages issue reports and updates task statuses.
- e. Admin oversees all operations, managing accounts, permissions, and system integrity.

- f.** The Booking class manages the reservation of classrooms, dorms, and facilities. It is linked to the Resource class, which defines the campus assets available (rooms, halls, buses, etc.).
- g.** Students and Faculty create booking requests.
- h.** Admin reviews and confirms them through the Approval relationship.
- i.** The system updates the booking status automatically in the Database entity after validation.
- j.** The IssueReport class allows users to submit problems related to campus facilities. Each issue is associated with a MaintenanceTask handled by the Maintenance user.
- k.** Maintenance staff update the issue's progress and resolution status.
- l.** Admin monitors maintenance completion and verifies closure reports.
This interaction ensures proper tracking of infrastructure and facility maintenance.
- m.** The Event class models all campus events created by Faculty or Admin. The Notification class ensures that all users receive timely alerts related to bookings, issues, and events.
- n.** Faculty creates events linked to the EventSchedule class.
- o.** The AuthenticationService class handles user login, token management, and security validation.
- p.** The MySQL database stores user profiles, bookings, issues, and events. It interacts with the system through API's
All class associations follow one-to-many and many-to-one relationships to represent how users, processes, and data components connect logically within the system.

Summary:

The class diagram establishes a strong connection between the SmartCampusSystem's functional modules and its underlying data structures. By defining inheritance, association, and relationships clearly, it provides a robust blueprint for implementation and ensures alignment with the DFDs and overall architecture.

VIII. Testing & Validation:

To ensure that the SmartCampus Management System functions as intended, a set of tests were conducted to detect errors, performance issues, and implementation flaws. These tests validate both functional and non-functional requirements to guarantee reliability, security, usability, and scalability. By executing targeted test cases, the team confirms that all modules of SmartCampus integrate smoothly and meet overall system objectives and user expectations.

A. Test Suite for Functional Requirements

Table 01: Login with valid credentials

Project Name: Smart Campus Management System						
Test Case ID: T_01			Test Designed by: Nour Itani			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Authentication			Test Executed by: Lynn Akil			
Test Title: Login with valid credentials			Test Execution date: TBD			
Description: Verify that users can log in successfully using valid credentials for all roles (Student, Faculty, Admin, Maintenance).						
Pre-conditions: User accounts exist and are active.						
Dependencies: User authentication module; connection to user database; valid registered user accounts; stable internet access.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Navigate to the SmartCampus login page	N/A	Login page loads successfully			
2	Enter a valid registered email address in the “Email” field	Email : student1@lau.edu.lb	Email field accepts input			
3	Enter the correct password in the “Password” field	Password: Student123	Password field accepts masked input			
4	Click the “Login” button	N/A	System processes authentication request			
5	Wait for dashboard to load	N/A	System authenticates and redirects users to correct role-based dashboard			
Post-conditions: Active user session created.						

Table 02: Login with invalid credentials

Project Name: Smart Campus Management System						
Test Case ID: T_02			Test Designed by: Nour Itani			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Authentication			Test Executed by: Lynn Akil			
Test title: Invalid login attempt			Test Execution date: TBD			
Description: Ensure the system blocks login when incorrect credentials are entered.						
Pre-conditions: User account exists.						
Dependencies: Authentication service and credential validation logic; error-handling mechanism for failed logins; connection to user database.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Navigate to the login page	N/A	Login page loads successfully			
2	Enter a valid email but incorrect password	Email: student1@lau.edu.lb Password: WrongPass	System recognizes invalid credentials			
3	Click on the “Login” button	N/A	Error message “Invalid credentials” displayed			
4	Observe system message	N/A	User remains on login page, no session created			
Post-conditions: No session created.						

Table 03: Forgot password

Project Name: Smart Campus Management System						
Test Case ID: T_03			Test Designed by: Nour Itani			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/1/2025			
Module Name: Authentication			Test Executed by: Lynn Akil			
Test Title: Password reset			Test Execution date: TBD			
Description: Verify password recovery process via registered email.						
Pre-conditions: User registered with a valid email.						
Dependencies: Active user authentication module; email verification and password reset service; secure token generation; SMTP/email delivery system.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Open the SmartCampus login page	N/A	Login page loads			
2	Click on “Forgot Password?” link	N/A	System navigates to password reset page			
3	Enter a registered email address	Email : student1@lau.edu.lb	System accepts input and validates email			
4	Click “Send Reset Link”	N/A	“Password reset email sent” confirmation displayed			
5	Check email inbox for reset link	N/A	Reset email received successfully			
Post-conditions: Password updated after reset link used.						

Table 04: Create new account

Project Name: Smart Campus Management System						
Test Case ID: T_04			Test Designed by: Nour Itani			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/1/2025			
Module Name: Authentication			Test Executed by: Lynn Akil			
Test Title: Account creation			Test Execution date: TBD			
Description: Verify successful creation of new user with valid data.						
Pre-conditions: User not previously registered.						
Dependencies: User registration and authentication module; database write permissions; input validation logic (email, phone, password format); stable internet connection.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Navigate to the login page	N/A	Login page loads			
2	Click on the “Sign Up” button	N/A	Sign-up form appears			
3	Enter full name, valid email, phone, and home address	Full name: John Dock Email: johndock@gmail.com Phone: 89-012-344 Address: LAU Ciella Dorm	Fields accept valid user data			
4	Create a password that meets requirements	Password: Joey39310;	System accepts strong password			
5	Confirm new password	Password confirmation: Joey39310;	Passwords match successfully			
6	Click “Sign Up”	N/A	Account created; “Registration successful” message displayed			
Post-conditions: New user can log in.						

Table 05: Book available resource

Project Name: Smart Campus Management System						
Test Case ID: T_05			Test Designed by: Nour Itani			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Booking			Test Executed by: Lynn Akil			
Test Title: Booking of available resource			Test Execution date: TBD			
Description: Verify users can book available classroom or room.						
Pre-conditions: User logged in; slot available.						
Dependencies: User authentication and session management; booking module; room availability database; transaction and data-update logic.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in to SmartCampus	Email: student1@lau.edu.lb Password: Student123	Dashboard loads successfully			
2	Navigate to the “Booking” module	N/A	Booking page opens			
3	Select an available room and time slot	Room: Library Room A Time: 10 AM – 12 PM	Room availability confirmed			
4	Click the “Book” button	N/A	System sends booking request			
5	Confirm booking in pop-up dialog	N/A	Confirmation message displayed and booking added to dashboard			
Post-conditions: Slot marked unavailable for others.						

Table 06: Prevent double booking

Project Name: Smart Campus Management System						
Test Case ID: T_06			Test Designed by: Nour Itani			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Booking			Test Executed by: Lynn Akil			
Test Title: Double-booking prevention			Test Execution date: TBD			
Description: Ensure two users cannot book the same resource/time slot.						
Pre-conditions: One slot already booked.						
Dependencies: Authentication and booking modules; real-time database validation; concurrency control to check occupied slots; notification/error-handling module.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as User A and book Room A (10 – 12)	User A: student1@lau.edu.lb Password: Student123 Room: A Time: 10–12 AM	Booking confirmed successfully			
2	Log in as User B and attempt to book the same slot	User B: student2@lau.edu.lb Password: Student222 Room: A Time: 10–12 AM	System rejects second booking			
3	Observe system response	N/A	Message “Slot unavailable” displayed; first booking remains valid			
Post-conditions: First booking remains valid.						

Table 07: Cancel existing booking

Project Name: Smart Campus Management System						
Test Case ID: T_07			Test Designed by: Lynn Akil			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/1/2025			
Module Name: Booking			Test Executed by: Nour Itani			
Test Title: Cancel booking			Test Execution date: TBD			
Description: Verify user can cancel an active booking						
Pre-conditions: User has confirmed booking						
Dependencies: Authentication and authorization modules; booking and database management system; user’s active booking record; confirmation/cancellation workflow enabled.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as a student with an active booking	Email: student1@lau.edu.lb Password: Student123	User successfully logs in and dashboard loads			
2	Open the “My Bookings” page	N/A	List of current bookings is displayed			
3	Select the desired booking from the list	Booking ID: B123	Booking details load correctly			
4	Click the “Cancel” button	N/A	System prompts a cancellation confirmation dialog			
5	Confirm cancellation in dialog box	N/A	Booking is deleted; success message displayed; slot becomes available again			
Post-conditions: Booking record updated.						

Table 08: Waitlist unavailable resource

Project Name: Smart Campus Management System						
Test Case ID: T_08			Test Designed by: Lynn Akil			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/1/2025			
Module Name: Booking			Test Executed by: Nour Itani			
Test Title: Waitlist full slot			Test Execution date: TBD			
Description: Ensure user is added to waitlist if slot unavailable.						
Pre-conditions: All slots booked.						
Dependencies: Authentication and booking modules; waitlist management logic; notification and alert service; database integrity for unavailable slots.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as a student	Email: student5@lau.edu.lb Password: Stud555	Student dashboard loads successfully			
2	Attempt to book a room with no available slots	Room: Library B Time: 2 PM – 4 PM	System displays “Slot full” message			
3	Accept the system’s offer to join the waitlist	N/A	User is added to waitlist and confirmation banner appears			
4	Check notification confirmation	N/A	Notification confirming waitlist addition is received			
Post-conditions: Waitlist entry created.						

Table 09: Submit issue report

Project Name: Smart Campus Management System						
Test Case ID: T_09			Test Designed by: Lynn Akil			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Issue Reporting			Test Executed by: Nour Itani			
Test Title: Submit issue report			Test Execution date: TBD			
Description: Verify issue reporting with valid data.						
Pre-conditions: User has logged in.						
Dependencies: Authentication module; issue reporting and database modules; validation for input fields; connection to the backend for data submission.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as a user	Email: student1@lau.edu.lb Password: Student123	User successfully logs in			
2	Navigate to the “Report Issue” section	N/A	Issue reporting form opens			
3	Enter issue title and description	Title: Projector Not Working Description: Screen flickers when turned on	Form fields accept valid text input			
4	Click “Submit”	N/A	System saves issue report and shows confirmation			
5	View confirmation message	N/A	“Issue submitted successfully” message appears and Issue ID is generated			
Post-conditions: Issue ID generated.						

Table 10: Attach file to issue

Project Name: Smart Campus Management System						
Test Case ID: T_10			Test Designed by: Lynn Akil			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/1/2025			
Module Name: Issue Reporting			Test Executed by: Nour Itani			
Test Title: Attachment upload			Test Execution date: TBD			
Description: Verify user can attach image to issue.						
Pre-conditions: User has logged in.						
Dependencies: Authentication and issue submission modules; file upload handler; secure storage path and validation for file types and sizes.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as a user	Email: student1@lau.edu.lb Password: Student123	User logged in successfully			
2	Go to “Report Issue” page	N/A	Issue submission form displayed			
3	Enter issue details	Title: Broken Chair in Lab 305	Fields accept data correctly			
4	Attach an image file to the form	File: chair.jpg Size: 800 KB	File uploads successfully without errors			
5	Click “Submit”	N/A	Issue and attachment are stored; confirmation displayed			
Post-conditions: Image linked to issue record.						

Table 11: Update issue status

Project Name: Smart Campus Management System						
Test Case ID: T_11			Test Designed by: Lynn Akil			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Maintenance			Test Executed by: Nour Itani			
Test Title: Issue status update			Test Execution date: TBD			
Description: Verify maintenance staff can change issue status.						
Pre-conditions: Issue assigned to staff.						
Dependencies: Authentication and role-based access modules; issue management system; database write permissions; logging and audit tracking functionality.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as maintenance staff	Email: maintenance1@lau.edu.lb Password: Maint123	Maintenance dashboard loads			
2	Open assigned issue list	N/A	List of assigned issues displayed			
3	Select a specific issue from list	Issue ID: #105	Issue details open successfully			
4	Change status to “In Progress”	Status: In Progress	System updates status instantly			
5	After repair, update status to “Closed”	Status: Closed	Issue marked “Closed”; confirmation message displayed			
Post-conditions: Log entry created.						

Table 12: Notify user on issue closure

Project Name: Smart Campus Management System						
Test Case ID: T_12			Test Designed by: Lynn Akil			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/1/2025			
Module Name: Notification System			Test Executed by: Nour Itani			
Test Title: Issue closure notification			Test Execution date: TBD			
Description: Verify automatic notification sent after issue resolved.						
Pre-conditions: Issue marked “Closed.”						
Dependencies: Authentication and notification modules; issue tracking system; automatic trigger for status change; user communication service (email or in-app notifications).						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as maintenance staff	Email: maintenance1@lau.edu.lb Password: Maint123	Staff logs in successfully			
2	Locate issue marked “Closed”	Issue ID: #105	Closed issue visible in list			
3	Verify that system automatically sends notification to reporting user	N/A	Notification triggered by backend process			
4	Log in as reporting user and check notifications tab	Email: student1@lau.edu.lb Password: Student123	Notification “Your issue #105 has been resolved” appears in user dashboard			
Post-conditions: Notification logged.						

Table 13: Chatbot valid response

Project Name: Smart Campus Management System						
Test Case ID: T_13			Test Designed by: Rami Assi			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/1/2025			
Module Name: AI Chatbot			Test Executed by: Rami Assi			
Test Title: Valid query response			Test Execution date: TBD			
Description: Verify chatbot gives correct replies to known campus queries.						
Pre-conditions: Chatbot trained with FAQs.						
Dependencies: User authentication and session modules; chatbot service and AI response engine; database of campus FAQs; stable connection between frontend chat UI and backend NLP logic.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in to SmartCampus	Email: student1@lau.edu.lb Password: Student123	Student logs in successfully and dashboard appears			
2	Navigate to the “Help / Chatbot” feature	N/A	Chatbot interface opens			
3	Type a valid campus-related question in the chat window	Question: “What are the library opening hours?”	Chatbot processes input			
4	Wait for and observe chatbot response	N/A	Chatbot displays accurate reply, e.g., “The library is open from 8 AM to 8 PM.”			
Post-conditions: Session logged.						

Table 14: Chatbot fallback response

Project Name: Smart Campus Management System						
Test Case ID: T_14			Test Designed by: Rami Assi			
Test Priority (Low/Med/High): Low			Test Designed date: 11/1/2025			
Module Name: AI Chatbot			Test Executed by: Rami Assi			
Test Title: Unknown query fallback			Test Execution date: TBD			
Description: Verify chatbot handles unrecognized input gracefully.						
Pre-conditions: Chatbot online.						
Dependencies: Authentication and chatbot modules; fallback and error-handling logic; internet connectivity for message processing; conversation logging service.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in to SmartCampus	Email: student2@lau.edu.lb Password: Student222	Student logs in successfully			
2	Open chatbot window	N/A	Chatbot interface loads			
3	Enter an unrecognized query	Question: “blablax99test nonsense input”	System attempts to interpret message			
4	Observe fallback message	N/A	Chatbot replies with default message: “I didn’t understand that. Please contact support.”			
Post-conditions: Error logged.						

Table 15: Faculty override student booking

Project Name: Smart Campus Management System						
Test Case ID: T_15			Test Designed by: Rami Assi			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Faculty Dashboard			Test Executed by: Rami Assi			
Test Title: Override booking			Test Execution date: TBD			
Description: Verify faculty can override student booking for class use.						
Pre-conditions: Booking exists.						
Dependencies: User authentication module; faculty role authorization; booking and scheduling modules; audit log system to record override actions.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as faculty	Email: faculty1@lau.edu.lb Password: Faculty123	Email: faculty1@lau.edu.lb Password: Faculty123			
2	Navigate to “Room Booking Management”	N/A	Booking management page loads			
3	Select a slot already booked by a student	Room: Science 201 Time: 9 – 10 AM	Existing student booking displayed			
4	Click “Override” to replace student’s booking	N/A	System prompts confirmation dialog			
5	Confirm override action	N/A	Faculty booking replaces student’s; confirmation message shown, and change logged			
Post-conditions: Updated ownership stored.						

Table 16: Faculty post announcement

Project Name: Smart Campus Management System						
Test Case ID: T_16			Test Designed by: Rami Assi			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/1/2025			
Module Name: Faculty Dashboard			Test Executed by: Rami Assi			
Test Title: Post announcement			Test Execution date: TBD			
Description: Verify faculty can post announcements visible to students.						
Pre-conditions: Faculty logged in.						
Dependencies: Authentication and role-based access control; announcement posting and storage modules; student dashboard integration for visibility; notification system for broadcast delivery.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as faculty	Email: faculty1@lau.edu.lb Password: Faculty123	Faculty home page loads			
2	Open the “Announcements” page	N/A	Announcement creation form displayed			
3	Type announcement message	Message: “Class shifted to Room 205 at 2 PM.”	Text box accepts input			
4	Click “Post”	N/A	Announcement published and visible on student dashboards			
Post-conditions: Message stored.						

Table 17: View faculty announcements

Project Name: Smart Campus Management System						
Test Case ID: T_17			Test Designed by: Assil Halawi			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/1/2025			
Module Name: Student Dashboard			Test Executed by: Abdallah ElRizz			
Test Title: View announcements			Test Execution date: TBD			
Description: Verify students can view announcements posted by faculty						
Pre-conditions: Announcement exists						
Dependencies: User authentication and session management; announcement retrieval module; database containing active announcements; student dashboard display integration.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as a student	Email: student1@lau.edu.lb Password: Student123	Student dashboard loads successfully			
2	Navigate to the “Announcements” page	N/A	Announcements page opens			
3	Review visible announcements	N/A	Recently posted announcements appear with timestamp			
4	Open announcement details (if available)	Announcement: “Class shifted to Room 205 at 2 PM.”	Full announcement content is displayed correctly			
Post-conditions: Cached for quick access						

Table 18: Update facility status

Project Name: Smart Campus Management System						
Test Case ID: T_18			Test Designed by: Assil Halawi			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Maintenance			Test Executed by: Abdallah ElRizz			
Test Title: Set facility under maintenance			Test Execution date: TBD			
Description: Verify maintenance staff can update facility status						
Pre-conditions: Facility listed in database						
Dependencies: Authentication and maintenance role authorization; facility management module; database update permissions; logging and status-tracking functionality.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as maintenance staff	Email: maintenance1@lau.edu.lb Password: Maint123	Maintenance dashboard loads			
2	Open “Facility Management” module	N/A	List of campus facilities displayed			
3	Select a facility currently active	Facility: Gym Hall	Facility information loads			
4	Change status to “Under Maintenance”	New Status: “Under Maintenance”	Status updated in database and reflected in system			
5	Save changes	N/A	Success message “Status updated successfully” displayed			
Post-conditions: Facility hidden from booking						

Table 19: Reopen Maintained Facility

Project Name: Smart Campus Management System						
Test Case ID: T_19			Test Designed by: Abdallah Elrizz			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/3/2025			
Module Name: Maintenance			Test Executed by: Assil Halawi			
Test Title: Reopen facility			Test Execution date: TBD			
Description: Verify staff can restore facility to “Active.”						
Pre-conditions: Facility previously under maintenance.						
Dependencies: Authentication and role-based access control; facility management system; update and confirmation workflow; audit log to record facility state changes.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as maintenance staff	Email: maintenance1@lau.edu.lb Password: Maint123	Staff logs in successfully			
2	Navigate to “Facility Management	N/A	Facility list opens			
3	Select facility marked “Under Maintenance”	Facility: Gym Hall	Facility details load			
4	Change status to “Active”	New Status: Active	Status change accepted by system			
5	Save and confirm	N/A	Facility restored to active state; confirmation message appears			
Post-conditions: Database updated.						

Table 20: Approve or Reject Booking

Project Name: Smart Campus Management System						
Test Case ID: T_20			Test Designed by: Abdallah Elrizz			
Test Priority (Low/Med/High): High			Test Designed date: 11/3/2025			
Module Name: Admin Dashboard			Test Executed by: Assil Halawi			
Test Title: Approve/Reject bookings			Test Execution date: TBD			
Description: Verify admin can process pending booking requests.						
Pre-conditions: Pending requests exist.						
Dependencies: Authentication and admin privilege validation; booking approval workflow; database transaction handling; notification service to inform users of approval or rejection.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as admin	Email: admin1@lau.edu.lb Password: Admin123	Admin dashboard opens			
2	Navigate to “Pending Bookings”	N/A	List of pending booking requests displayed			
3	Select a booking request	Request ID: #554	Booking details appear			
4	Choose “Approve” or “Reject”	Action: Approve / Reject	System processes request and updates status			
5	Confirm action and verify notification	N/A	Confirmation message shown; user receives booking status notification			
Post-conditions: Status updated.						

Table 21: Dorm Booking and Cancellation

Project Name: Smart Campus Management System						
Test Case ID: T_21			Test Designed by: Abdallah Elrizz			
Test Priority (Low/Med/High): High			Test Designed date: 11/3/2025			
Module Name: Student Housing			Test Executed by: Assil Halawi			
Test Title: Dorm booking/cancellation			Test Execution date: TBD			
Description: Verify students can book and cancel dorm rooms.						
Pre-conditions: Available dorms exist.						
Dependencies: Authentication and student role validation; housing management module; booking and cancellation workflow; dorm database for room availability updates.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as student	Email: student3@lau.edu.lb Password: Stud333	Student dashboard loads successfully			
2	Go to “Housing” module	N/A	Housing page opens and available dorms displayed			
3	Select available dorm room	Dorm: B-203 Duration: 1 semester	Room details appear and “Book” option enabled			
4	Click “Book”	N/A	System processes booking request			
5	Confirm booking details	N/A	Confirmation message “Dorm booked successfully” displayed			
Post-conditions: Room availability updated.						

Table 22: Transportation Schedule and Booking

Project Name: Smart Campus Management System						
Test Case ID: T_22			Test Designed by: Abdallah Elrizz			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/3/2025			
Module Name: Transportation			Test Executed by: Assil Halawi			
Test Title: Bus reservation			Test Execution date: TBD			
Description: Verify student can view routes and reserve seats.						
Pre-conditions: Routes defined in system.						
Dependencies: Authentication module; transportation scheduling and reservation subsystem; route and timing database; notification/confirmation service for reservations.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as student	Email: student4@lau.edu.lb Password: Stud444	Student dashboard displayed			
2	Navigate to “Transportation” module	N/A	Transportation schedule page opens			
3	View available routes and timings	N/A	All routes and times listed correctly			
4	Select desired route and time	Route: LAU → Downtown Time: 8:00 AM	Selected route highlighted			
5	Click “Reserve”	N/A	System sends reservation request			
Post-conditions: Reservations saved.						

Table 23: Event Ticket Reservation

Project Name: Smart Campus Management System						
Test Case ID: T_23			Test Designed by: Abdallah Elrizz			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/3/2025			
Module Name: Events			Test Executed by: Assil Halawi			
Test Title: Event booking			Test Execution date: TBD			
Description: Verify users can reserve tickets for campus events.						
Pre-conditions: Event available.						
Dependencies: Authentication and session management; events management module; database of upcoming events; booking confirmation and notification logic.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in to SmartCampus	Email: student5@lau.edu.lb Password: Stud555	Student dashboard loads			
2	Navigate to the “Events” section	N/A	List of campus events displayed			
3	Select an upcoming event	Event: Career Fair 2025	Event details page opens			
4	Click “Reserve Ticket”	N/A	Ticket reservation request sent			
5	Confirm reservation	N/A	“Your ticket has been reserved” message displayed			
Post-conditions: Ticket stored in dashboard.						

Table 24: Admin View Activity Logs

Project Name: Smart Campus Management System						
Test Case ID: T_24			Test Designed by: Abdallah Elrizz			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/3/2025			
Module Name: Admin Dashboard			Test Executed by: Assil Halawi			
Test Title: View activity logs			Test Execution date: TBD			
Description: Verify admin can view recorded system activity for auditing.						
Pre-conditions: Logging system enabled.						
Dependencies: Authentication and admin role authorization; system logging module; activity log database; report generation and filtering functions.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as admin	Email: admin1@lau.edu.lb Password: Admin123	Admin dashboard opens successfully			
2	Navigate to “System Activity Logs”	N/A	Log management interface opens			
3	Apply filters to view logs by user or date range	User: student1@lau.edu.lb Date Range: 01/10/2025 – 31/10/2025	Filter criteria applied correctly			
4	Generate report	N/A	Report generated with filtered actions			
5	Review log entries	N/A	User actions with timestamps displayed accurately on screen			
Post-conditions: Audit verified.						

B. Test Suite for Non-Functional Requirements

Table 25: System response time

Project Name: Smart Campus Management System						
Test Case ID: T_25			Test Designed by: Abdallah Elrizz			
Test Priority (Low/Med/High): High			Test Designed date: 11/3/2025			
Module Name: Performance			Test Executed by: Assil Halawi			
Test Title: View system response time			Test Execution date: TBD			
Description: Ensure that pages load within 3 seconds during normal usage.						
Pre-conditions: Server online with ≤ 1 000 concurrent users.						
Dependencies: Active authentication and routing modules; optimized database queries; backend request-handling logic; network connectivity for real-time responses.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Launch SmartCampus web app in browser	Browser: Chrome v122	Home page loads within 3 seconds			
2	Log in with valid credentials	Email: student1@lau.edu.lb Password: Student123	Login completes within 3 seconds			
3	Navigate Dashboard → Booking → Profile	N/A	Each page transition ≤ 3 seconds			
4	Record average load time using Cypress timer	N/A	All measured times ≤ target threshold			
Post-conditions: Response-time metrics recorded for analysis						

Table 26: Load Handling

Project Name: Smart Campus Management System						
Test Case ID: T_26			Test Designed by: Abdallah Elrizz			
Test Priority (Low/Med/High): High			Test Designed date: 11/3/2025			
Module Name: Performance			Test Executed by: Assil Halawi			
Test Title: Load testing under heavy concurrency			Test Execution date: TBD			
Description: Verify system remains stable under 1 000 concurrent users.						
Pre-conditions: Stress-test environment configured.						
Dependencies: Authentication and session management; scalable backend architecture; database concurrency control; load-balancing and queue-handling logic						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Configure JMeter to simulate 1 000 virtual users	User IDs S001–S1000	Test setup completes without error			
2	Start load test (trigger login + booking actions)	Duration: 15 min	System remains responsive throughout			
3	Monitor memory usage on MySQL Workbench	N/A	Usage stays below 80 %			
4	Collect error rate and throughput results	N/A	Error rate < 2 %, average response ≤ 5 s			
Post-conditions: Load-test summary generated.						

Table 27: Data encryption

Project Name: Smart Campus Management System						
Test Case ID: T_27			Test Designed by: Abdallah Elrizz			
Test Priority (Low/Med/High): High			Test Designed date: 11/3/2025			
Module Name: Security			Test Executed by: Assil Halawi			
Test Title: Verify encryption in storage & transit			Test Execution date: TBD			
Description: Confirm user data is encrypted both in database and over network.						
Pre-conditions: User logged in.						
Dependencies: Authentication and secure login process; encryption and decryption layer in backend; secure database storage; HTTPS communication protocol.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in to SmartCampus	Email: student1@lau.edu.lb Password: Student123	User authenticated over HTTPS			
2	Capture network requests in DevTools	Protocol: HTTPS TLS 1.3	All traffic encrypted			
3	Inspect payload content	N/A	No plain-text credentials visible			
4	Use SQL queries to check stored password hash	Field: user.passwordHash	Data stored as SHA-256 hash			
Post-conditions: Encryption verified in logs.						

Table 28: Admin multi-factor authentication

Project Name: Smart Campus Management System						
Test Case ID: T_28			Test Designed by: Assil Halawi			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Security			Test Executed by: Abdallah Elrizz			
Test Title: Verify MFA for admin login			Test Execution date: TBD			
Description: Ensure that admin users must successfully complete a second authentication factor before system access is granted						
Pre-conditions: Admin account exists and is active.						
Dependencies: MFA module enabled; User authentication and admin role modules; OTP generation and verification logic; secure session management; access-control validation.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Open SmartCampus Admin Login page	N/A	Login form loads			
2	Enter admin email and password	Email: admin1@lau.edu.lb Password: Admin123	System prompts for OTP			
3	Receive OTP via email/SMS from Railway Cloud Functions via logs	OTP: 654321 (Valid 60 s)	OTP delivered successfully			
4	Enter OTP into MFA prompt	N/A	Authentication validated			
5	Verify access granted post-OTP	N/A	Admin dashboard loads; token stored Railway MySQL			
Post-conditions: MFA token recorded.						

Table 29: Access control

Project Name: Smart Campus Management System						
Test Case ID: T_29			Test Designed by: Assil Halawi			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Security			Test Executed by: Abdallah Elrizz			
Test Title: Role-based access control			Test Execution date: TBD			
Description: Confirm that each user role can access only authorized modules and pages						
Pre-conditions: Active accounts for Admin, Faculty, Student, Maintenance						
Dependencies: Authentication module; role-based authorization logic; user session management; page and resource permission mapping						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Log in as student	Email: student1@lau.edu.lb Password: Stud123	Student dashboard loads			
2	Attempt to access Admin Dashboard URL	/admin/manage-users	System returns “Access Denied”			
3	Log in as maintenance staff and access faculty page	Email: maint1@lau.edu.lb Password: Maint123 URL: /faculty/grades	Unauthorized access blocked; redirected to home			
4	Check Railway log trail	N/A	Unauthorized attempt logged			
Post-conditions: Unauthorized attempt logged						

Table 30: UI consistency

Project Name: Smart Campus Management System						
Test Case ID: T_30			Test Designed by: Assil Halawi			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/1/2025			
Module Name: Usability			Test Executed by: Abdallah ElRizz			
Test Title: Check interface consistency			Test Execution date: TBD			
Description: Ensure consistent colors, fonts, and layout across the whole system						
Pre-conditions: Web UI fully deployed						
Dependencies: Frontend layout framework; shared CSS design system; component rendering module; design validation guidelines						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Navigate through Home, Booking, Issues, Events, and Profile pages	N/A	Each page renders same layout			
2	Capture screenshots via Cypress for comparison	N/A	Images match baseline design			
3	Compare colors, fonts with Figma UI kit	Font: Roboto 14 pt Color: #1A73E8 (will change)	All elements meet design standards			
Post-conditions: Visual consistency report exported and approved by QA (Quality Assurance) team						

Table 31: Mobile responsiveness

Project Name: Smart Campus Management System						
Test Case ID: T_31			Test Designed by: Assil Halawi			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/1/2025			
Module Name: Usability			Test Executed by: Abdallah Elrizz			
Test Title: Verify responsive layout			Test Execution date: TBD			
Description: Confirm that the Smart Campus System renders properly on various mobile and tablet devices without layout distortion						
Pre-conditions: Webpage deployed and reachable on a mobile device						
Dependencies: Frontend layout engine; responsive CSS grid and media-query rules; cross-platform rendering logic; dynamic resizing functions						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Open Smart Campus System on mobile and tablet devices	Devices: iPhone, Tablet, Samsung iPhone	Layout adapts to screen size			
2	Navigate through Dashboard, Bookings, and Profile	N/A	All pages fit viewport without overflow			
3	Check for alignment, text overflow, and scroll behavior	Screen sizes: 390×844 1280×800	No horizontal scroll bars appear			
4	Switch to landscape orientation and re-check layout	N/A	Layout remains intact; responsive CSS active			
Post-conditions: Responsiveness confirmed, report archived in QA repository						

Table 32: Daily backups

Project Name: Smart Campus Management System						
Test Case ID: T_32			Test Designed by: Assil Halawi			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Reliability & Recovery			Test Executed by: Abdallah Elrizz			
Test Title: Automatic daily backup			Test Execution date: TBD			
Description: Validate that automated daily database backups are executed and stored securely						
Pre-conditions: Backup scheduler enabled in Railway Cloud Functions						
Dependencies: Database management and storage modules; automated backup scheduler; file-integrity verification logic; administrator privileges for restore access						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Trigger daily backup schedule via Railway Functions	N/A	Backup process initiated			
2	Wait 24 hours for automatic backup	N/A	New backup file created in Storage			
3	Check Storage bucket for backup	Backup folder: /backups/	File exists with current date			
4	Verify timestamp and file size	Expected > 10 MB	Backup valid and complete			
Post-conditions: Backup integrity verified						

Table 33: System Recovery

Project Name: Smart Campus Management System						
Test Case ID: T_33			Test Designed by: Assil Halawi			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Reliability & Recovery			Test Executed by: Abdallah Elrizz			
Test Title: Restore from backup			Test Execution date: TBD			
Description: Backup and restore subsystem; admin authorization; data-validation and error-recovery logic; connection to backup storage.						
Pre-conditions: Latest backup available						
Dependencies: Railway Cloud Functions, Postman, Cypress (for post-restore UI validation).						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Delete sample record from Railway	Record: Issue #105	Record removed			
2	Trigger restore function via Postman API	Backup file: backup_20251101.json	Restore initiated			
3	Execute recovery process	N/A	Data restored within 24 h			
4	Verify restored data in UI via Cypress	N/A	Deleted record reappears successfully			
Post-conditions: System status flag set to “Operational” in Admin Panel.						

Table 34: System uptime

Project Name: Smart Campus Management System						
Test Case ID: T_34			Test Designed by: Assil Halawi			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Availability			Test Executed by: Abdallah Elrizz			
Test Title: Verify uptime percentage			Test Execution date: TBD			
Description: Ensure uptime ≥ 99.5 % excluding maintenance						
Pre-conditions: Monitoring active for ≥ 1 month						
Dependencies: Hosting environment; monitoring and logging modules; failover and redundancy logic; network infrastructure stability.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Access hosting monitor dashboard	Tool: UptimeRobot / Railway log Status	Dashboard loads uptime report			
2	Retrieve uptime report for the last 30 days	Observation period: December 1 – December 31 2025	Report downloaded			
3	Review any downtime logs	N/A	Total downtime ≤ 3 h			
4	Calculate total uptime percentage	Target uptime: ≥ 99.5 %	Requirement met and recorded			
Post-conditions: Results archived						

Table 35: Scalability test

Project Name: Smart Campus Management System						
Test Case ID: T_35			Test Designed by: Assil Halawi			
Test Priority (Low/Med/High): High			Test Designed date: 11/1/2025			
Module Name: Performance & Capacity			Test Executed by: Abdallah Elrizz			
Test Title: Verify scalability under user growth			Test Execution date: TBD			
Description: Confirm system stability with 1500 users						
Pre-conditions: Auto-scaling enabled on Railway						
Dependencies: Authentication and user-session modules; autoscaling backend configuration; load-distribution logic; real-time monitoring system.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Configure load test for 1500 simulated users	Users: U001–U1500	Load test initialized			
2	Execute mixed operations (login, booking, reporting issues).	Load duration: 20 minutes	System stable throughout			
3	Monitor CPU usage and response time	CPU ≤ 80 % Response ≤ 7 s	Performance within limits			
4	Review autoscaling logs	N/A	Scaling triggered and handled load efficiently			
Post-conditions: Metrics recorded for optimization						

Table 36: Cross-browser compatibility

Project Name: Smart Campus Management System						
Test Case ID: T_36			Test Designed by: Assil Halawi			
Test Priority (Low/Med/High): Medium			Test Designed date: 11/1/2025			
Module Name: Usability / Compatibility			Test Executed by: Abdallah Elrizz			
Test Title: Browser compatibility test			Test Execution date: TBD			
Description: Ensure all features work on major browsers						
Pre-conditions: Browsers installed (Chrome, Edge, Firefox, Safari)						
Dependencies: Frontend rendering engine; UI and JavaScript compatibility layer; shared CSS framework; authentication and routing modules.						
Step	Test Steps	Test Data	Expected Result	Actual Results	Status	Notes
1	Launch SmartCampus on different browsers	Browsers: Chrome v122, Edge v120, Firefox v118, Safari v17	All browsers load homepage			
2	Perform login and booking operations on each	student1@lau.edu.lb Password: Student123	All operations succeed			
3	Observe UI rendering and functionality	N/A	Identical appearance and behavior			
4	Review console logs for errors	N/A	No JavaScript or CSS errors detected			
Post-conditions: Compatibility results documented, QA report generated and signed off by tester.						

IX. System Evolution

To ensure the Smart Campus System stays reliable, scalable, and meets the needs of students, faculty, and staff, the system will evolve with these strategies:

- The system will gather regular feedback from users through surveys and analytics to identify areas for improvement.
- System updates and improvements will be scheduled during low-traffic periods to minimize disruptions.
- The system will allow new services to be added without major changes.
- The platform will scale to support more users, adding new campuses and resources as needed.
- The system will work on both cloud (since everything is changing to the cloud) and hybrid setups, adapting to growing storage and computing needs.
- Smart Campus will connect with existing university tools (portal, student databases) to provide a seamless experience.
- Future integrations will include payment systems.
- The system will incorporate advanced AI for better predictions and resource management.
- The system will support new technology, like IoT sensors for better classroom management and energy use.

X. Appendices

A. Minimum Hardware Requirements

The following outlines the minimum hardware requirements necessary for the Smart Campus Management System to function properly on various platforms:

Mobile Application Requirements:

For Android:

CPU: ARMv7 or x86 | Recommended: Snapdragon 660 or better

RAM: 2 GB | Recommended: 3 GB or more

Storage: 100 MB of free space | Recommended: 500 MB free

Operating System: Android 7.0+ | Recommended: Latest Android version

For iOS/iPadOS:

Device: iPhone, iPad, or iPod touch | Recommended: iPhone 8 or newer

RAM: 2 GB | Recommended: 3 GB or more

Storage: 100 MB of free space | Recommended: 500 MB free

Operating System: iOS 15 or later | Recommended: Latest iOS version

B. Application Dependencies

The Smart Campus Management System integrates with several external technologies to enhance its functionality and ensure smooth operation. These dependencies are crucial for user authentication, data management, cloud hosting, and other essential system features:

1. User Authentication:

- **MySQL-Based Authentication:**

User authentication is handled directly through the MySQL database, where all user credentials are securely stored. Passwords are hashed using bcrypt, and access control is enforced using JWT (JSON Web Tokens) to ensure secure and role-based access to system features.

2. Version Control:

- **GitHub:** GitHub manages the source code for version control and collaboration among developers. GitHub Actions automates the CI/CD pipeline for seamless testing and deployment.

3. Backend Hosting:

- **Railway:**

The backend server, developed using Node.js and Express.js, is hosted on Railway. This allows scalable deployment, environment variable management, and seamless integration with the hosted MySQL database.

4. Database Management:

- **MySQL (Railway):**

The system uses a relational MySQL database, also hosted on Railway. It stores structured data such as users, bookings, issues, events, notifications, tutoring sessions, and other system modules. SQL queries manage all data retrieval, insertions, updates, and role-based interactions.

5. Database Management:

- **MySQL (Railway):**

The system uses a relational MySQL database, also hosted on Railway. It stores structured data such as users, bookings, issues, events, notifications, tutoring sessions, and other system modules. SQL queries manage all data retrieval, insertions, updates, and role-based interactions.

6. Frontend Hosting:

- The frontend interface—built with HTML, CSS, and JavaScript—is deployed on Vercel. This provides fast global delivery, automatic deployments from GitHub, and a public URL for users to access the platform.

C. Database Schemas

1. USER

Represents any person using the system (student, faculty, maintenance, admin).

- **user_id:** Unique identifier for each user.
- **first_name:** User's first name.
- **last_name:** User's last name.
- **email:** University email address.
- **phone_number:** Contact number.
- **role:** User role (student, faculty, maintenance, admin).
- **department_id:** References department the user belongs to.
- **created_at:** Registration date.
- **status:** Account status (active, suspended, deactivated).

2. DEPARTMENT

Academic or administrative department.

- **department_id:** Unique identifier.
- **name:** Department name.

3. RESOURCE

Any reservable campus resource (room, lab, sports hall, library seat, computer, equipment).

- **resource_id:** Unique identifier.
- **resource_type:** Type (study room, lab, sports_hall, library_seat, computer, equipment).
- **name:** Resource label.
- **building:** Building name.
- **room_number:** Room number (if applicable).
- **capacity:** Max occupants
- **status:** active, under_maintenance, closed.

4. BOOKING_REQUEST

Faculty or student requests to reserve a resource (need approval from admin).

- **request_id:** Unique identifier.
- **user_id:** Who requested.

- **resource_id:** Requested resource.
- **start_time:** Start of reservation.
- **end_time:** End of reservation.
- **purpose:** Reason (lecture, exam, club, sports, study ...).
- **priority:** student or faculty.
- **status:** pending, approved, rejected, canceled.
- **reviewed_by_admin_id:** Admin who approved/rejected.
- **reviewed_at:** Decision timestamp.

5. BOOKING

Approved and active reservations (after approval by admin).

- **booking_id:** Unique identifier.
- **request_id:** Source request.
- **resource_id:** Reserved resource.
- **user_id:** Requester/owner.
- **start_time:** Start of reservation.
- **end_time:** End of reservation.
- **status:** active, expired, cancelled, reassigned...
- **created_at:** Creation time.

6. EVENT

Academic events, seminars, workshops.

- **event_id:** Unique identifier.
- **organizer_user_id:** Faculty or dept organizer.
- **title:** Event title.
- **description:** Event details.
- **resource_id:** Venue (optional if off-campus).
- **start_time:** Start date/time.
- **end_time:** End date/time.
- **capacity:** Seat limit.
- **status:** published, canceled, completed.

7. EVENT_TICKET

Reservations for events.

- **ticket_id:** Unique identifier.
- **event_id:** Associated event.
- **user_id:** Attendee.
- **reserved_at:** Timestamp.
- **status:** reserved, canceled, checked_in.

8. DORM_UNIT

Dorm rooms/units.

- **dorm_id:** Unique identifier.
- **assigned_user_id:** Student having reserved the uni
- **building:** Dorm building.
- **unit_number:** Room number.
- **capacity:** Beds in unit.
- **gender_policy:** male, female, mixed (per campus policy).
- **status:** available, occupied, maintenance, closed.
- **start_date:** Move-in.
- **end_date:** Move-out.

9. TRANSPORTATION_ROUTE

Off-campus university bus route and On-campus shuttle reservation.

- **route_id:** Unique identifier for each route.
- **route_name:** Name of the bus route.
- **start_point:** Departure Location
- **end_point:** Destination
- **capacity:** Seats per trip.
- **reserved_seats:** Seats already taken
- **status:** active, paused.

10. PARKING_SPACE

Parking slots.

- **parking_space_id:** Unique identifier.
- **lot_name:** Parking lot name.
- **spot_number:** Spot identifier.
- **user_id:** Reserving user.

- **type:** student, faculty/admin
- **status:** available, reserved, maintenance.

11. ISSUE_REPORT

Facility/IT/environmental issues.

- **issue_id:** Unique identifier.
- **reporter_user_id:** Who reported.
- **category:** maintenance, IT, environmental, general.
- **location:** Building/room/area.
- **description:** Problem details.
- **priority:** low, medium, high, critical.
- **status:** pending, in_progress, resolved, closed.
- **created_at:** Report time.
- **file_url:** Storage path (attachement, optional)

12. ISSUE_ACTIVITY

Update on the issue report

- **activity_id:** Unique identifier.
- **issue_id:** Related issue.
- **actor_user_id:** Staff updating.
- **action:** assign, start, update, add_note, add_photo, resolve, close, reassign.
- **note:** Optional text.
- **time_of_activity:** Timestamp.

13. CLEANING_TASK

Daily cleaning schedules and status.

- **task_id:** Unique identifier.
- **resource_id:** Area/room to clean.
- **assigned_to:** Staff user_id.
- **scheduled_for:** Date/time.
- **status:** pending, in_progress, completed.
- **notes:** Comments.

14. TUTORING_SESSION

Tutoring linked to courses.

- **session_id:** Unique identifier.
- **created_by_user_id:** Faculty/TA organizer.
- **course_code:** Course identifier (optional).
- **resource_id:** Room.
- **start_time:** Start.
- **end_time:** End.
- **capacity:** Seats.

15. STUDENT_EMPLOYMENT

Represents jobs available for students within the campus.

- **job_id:** Unique identifier for the job.
- **job_title:** The title of the job (“Library Assistant”, “Campus Tutor” ...).
- **department_id:** Department or office offering the job (links to DEPARTMENT table).
- **description:** Brief job description and responsibilities.
- **salary:** Compensation (hourly rate).
- **hours_per_week:** Expected number of working hours per week.
- **status:** active, closed (whether the job is available or no longer accepting applications).
- **posted_at:** Timestamp when the job was posted.
- **application_deadline:** Deadline for students to apply.
- **supervisor_id:** Faculty or staff member supervising the job.

16. EMPLOYMENT_APPLICATION

Represents an application made by a student for a campus job.

- **application_id:** Unique identifier for the application.
- **job_id:** Job the student is applying for
- **user_id:** Student applying for the job
- **application_date:** Date when the application was submitted

17. NOTIFICATION

In-app/email notifications.

- **notification_id:** Unique identifier.
- **user_id:** Recipient.

- **type:** booking, issue, event, transport, admin, system.
- **message:** Text content.
- **sent_at:** Timestamp.
- **notification_method:** in-app, email, sms.

D. System Implementation & Deployment

This appendix summarizes the final implementation of the SmartCampus Management System completed in Phase IV. The system was fully developed, deployed, and tested based on the requirements outlined in Phases I–III. The final solution reflects a complete client–server architecture with a functional backend, dynamic user interfaces, and a relational database.

1. System Implementation Overview

The SmartCampus platform was implemented using a three-tier MVC architecture:

Frontend (Presentation Layer)

- Technologies: HTML, CSS, JavaScript
- Four role-based dashboards: Student, Faculty, Maintenance, Admin
- Features include room booking, issue reporting, event viewing, notifications, and resource management...

Backend (Application Layer)

- Built using Node.js + Express.js
- RESTful API endpoints handle authentication, bookings, issues, events, tutoring sessions, lost & found, and admin functions
- JWT authentication and bcrypt password hashing ensure secure access
- Modular MVC structure: controllers, services, routes, and models

Database (Data Layer)

- MySQL database hosted on Railway
- Structured relational schema with main tables:
 - users, bookings, issues, events, lostFound, notifications, tutoringSessions, studentEmployment, facilityStatus, housing, parking, transportation
- SQL queries manage CRUD operations for all system features

2. Deployment Summary

The system was deployed using cloud hosting services to ensure accessibility and stability:

Frontend Deployment

- Hosted on Vercel
- Automatically builds from GitHub repository
- Provides a public URL for real-time access

Backend Deployment

- Hosted on Railway as a Node.js service
- Environment variables stored securely (DB credentials, JWT secret)

Database Hosting

- Railway MySQL instance
- Fully integrated with backend for live CRUD operations

Version Control

- Complete source code maintained on GitHub
- Separate repositories for frontend and backend
- Enables team collaboration and version tracking

3. System Workflow Summary

A typical end-to-end workflow in SmartCampus operates as follows:

1. User logs in → JWT token generated
2. User performs an action (e.g., book a room)
3. Frontend sends a JSON API request to backend
4. Backend validates token and business rules
5. SQL query executes on MySQL database
6. Backend returns confirmation or error
7. Dashboards update with new data

This workflow is repeated for all modules: bookings, issues, events, tutoring, lost & found, maintenance, and admin management.

4. Some Test Cases

Test Case 1 — Student Room Booking

- Action: Student submits booking request
- Expected Result: Booking is stored in MySQL and visible in Faculty/Admin dashboards
- Status: Passed

Test Case 2 — Faculty Approval

- Action: Faculty approves or rejects booking
- Expected Result: Status updated in database and reflected in student portal
- Status: Passed

Test Case 3 — Issue Reporting

- Action: Student submits an issue (AC, projector, etc.)
- Expected Result: Maintenance dashboard shows new issue with correct priority
- Status: Passed

Test Case 4 — Maintenance Progress Update

- Action: Maintenance updates issue status
- Expected Result: Student sees progress in real time
- Status: Passed

Test Case 5 — Authentication (Invalid Login)

- Action: Wrong password entered
- Expected Result: Backend rejects login with error
- Status: Passed

Test Case 6 — API Response Validation

- Action: Fetch bookings via REST API
- Expected Result: Valid JSON response with correct fields
- Status: Passed

6. Conclusion

The SmartCampus system was successfully implemented and deployed using modern web technologies. All core functionalities outlined in the earlier phases were fully realized, including multi-role dashboards, secure authentication, structured data management, and cloud deployment. The system meets the project requirements and provides a solid foundation for future enhancements such as push notifications, analytics dashboards, or a dedicated mobile application.

XI. Index

A:

Admin Operations – Page 29
AI Chatbot – Page 5
AJAX – Page 15
Analytics and Performance Monitoring – Page 41
API – Page 5
Application Dependencies – Page 40
Application Layer – Page 16
Appendices – Page 39

B:

Backend Hosting – Page 40
Booking and Resource Management – Page 36
Booking Request – Page 43
Bootstrap – Page 15

C:

Class Diagram Representation – Page 35
Client-Server Architecture – Page 14-19
Cloud-Based Deployment – Page 5
Context Diagram (DFD Level 0) – Page 32
Cypress – Page 16

D:

Data and Storage Components – Page 37
Data Encryption – Page 5
Data Layer – Page 17
Database Interaction – Page 20
Database Management – Page 40

Database Schemas – Page 42

Detailed Process Flow (DFD Level 1) – Page 33

DFD – Page 32

Disaster Recovery Requirements – Page 31

E:

Employment Application – Page 48

Event Management – Page 44

F:

Faculty Operations – Page 27

Fetch API – Page 15

Functional Requirements – Page 23

I:

Issue Reporting and Maintenance Operations – Page 36

L:

Layered Architecture – Page 14

M:

Maintenance & Facility Operations – Page 28

Minimum Hardware Requirements – Page 39

N:

Non-Functional Requirements – Page 31

Notification System – Page 30

P:

Payment Authentication – Page 40

Postman – Page 17

Presentation Layer – Page 15

Push Notifications – Page 41

S:

Security and Compliance – Page 30

Smart Campus Management System – Page 0

Software Engineering – Page 0

System Administrator Requirements – Page 13

System Architecture – Page 7-14

System Configuration & Monitoring – Page 30

System Evolution – Page 38

System Model – Page 32

T:

Transportation Routes – Page 45

Tutoring Session – Page 47

U:

User Authentication – Page 40

User Interaction – Page 19

User Management – Page 35

User Operations – Page 23

User Requirements – Page 9

V:

Version Control – Page 40